Chapter 48: The Rendering Engine

1. Overview

Every Angular template you write is thrown away before it reaches the browser. The compiler (`@angular/compiler`, invoked ahead-of-time by the CLI) turns your HTML-like template into a plain TypeScript function attached to the component's `ɵcmp` definition — a `ComponentDef`. That function is the component's **template function**, and it is the only thing the rendering engine (Ivy / render3) actually executes. There is no template string at runtime, no HTML parser, no virtual DOM diff. There is just a function that, when called, issues a sequence of **instructions** — `ɵɵelementStart`, `ɵɵtext`, `ɵɵproperty`, `ɵɵlistener`, `ɵɵadvance`, and dozens more — that either build DOM nodes the first time (**create mode**) or refresh already-built DOM nodes on every subsequent call (**update mode**).

This chapter is about that machinery: what the instruction set actually is, how the same template function runs in two different modes controlled by a bitmask, how `LView` / `TView` / `TNode` data structures back every component instance, how structural directives like `*ngIf`, `*ngFor`, `@if`, `@for` create and destroy **embedded views** anchored to comment nodes via `ViewContainerRef`, how dependency injection resolves differently at each node during rendering (the node injector tree vs the module injector tree), and finally how this rendering engine and the change detection engine are really the same walk — CD is not a separate system that "calls into" rendering, CD *is* the repeated invocation of the update-mode instructions.

Interviewers ask about this because it separates people who've used Angular from people who understand why Angular behaves the way it does — why `OnPush` skips subtrees, why `ngFor` `trackBy` matters, why moving a DOM node with a structural directive is cheap, why injecting a service in a component vs a directive on the same element can yield different instances, and why "the change detector runs the template again" is a more literal statement than most people realize.

2. Core Concepts

2.1 The compiled artifact: `ComponentDef` and the template function

Every component decorated with `@Component` compiles down to a static field on the class:

```
static ɵcmp = ɵɵdefineComponent({
  type: MyComponent,
  selectors: [["app-my"]],
  decls: 5,     // number of "slots" this view allocates in LView (nodes, pipes, i18n
blocks...)
  vars: 3,      // number of binding slots used for change-detection dirty checking
  template: function MyComponent_Template(rf: RenderFlags, ctx: MyComponent) {
    if (rf & RenderFlags.Create) {
      // create-mode instructions run ONCE per view instance
    }
    if (rf & RenderFlags.Update) {
      // update-mode instructions run on EVERY change detection pass
    }
  },
  // ...directives, pipes, etc.
});
```

`RenderFlags` is a bitmask enum: `Create = 0b01`, `Update = 0b10`. The **same function** is called twice conceptually — once with `Create` when the view is first instantiated, and thereafter with `Update` on every CD pass. In practice Angular calls it with `Create | Update` the very first time (build the nodes, then immediately populate their bindings), and with `Update` only afterward.

This dual-mode design is the crux of render3: **one function serves both jobs**, which is why the compiler emits `ɵɵadvance(n)` calls to move an internal cursor rather than re-addressing nodes by index — the same instruction stream structure is walked for creation and for refresh, just executing a different subset of instructions gated by `rf &`.

2.2 `decls` and `vars`

- `decls` (declarations) — the count of "slots" the view needs in its `LView` array: one per element, text node, container, pipe instance, i18n block, etc. This is fixed at compile time and drives `LView` array pre-allocation (`HEADER_OFFSET` + `decls`).
- `vars` (variables) — the count of binding slots reserved for the "no-change" tracking of dynamic values (interpolations, property bindings, pipe pure-tracking). Each binding gets a slot to memoize its previous value against, which is how `ɵɵproperty` / `ɵɵtextInterpolate` avoid touching the DOM when the value hasn't changed (`bindingUpdated` checks).

2.3 The render3 instruction set

Instructions are just functions in `@angular/core`, prefixed `ɵɵ` to mark them as private/internal API used only by generated code. The important families:

Node creation (create mode only)

- `ɵɵelementStart(index, tagName, ...)` / `ɵɵelementEnd()` — create a host element, push a `TNode`, descend into it as the "current" node for child instructions; `elementEnd` pops back up.
- `ɵɵelement(index, tagName, ...)` — shorthand for a self-closing/no-children element (start+end fused).
- `ɵɵtext(index, value?)` — create a `Text` DOM node.
- `ɵɵtemplate(index, templateFn, decls, vars, tagName, attrs)` — create an **embedded view's anchor**: a comment node (`<!--container-->`) plus a registered `LContainer`, used for `*ngIf`, `*ngFor`, `<ng-template>`, `@if`, `@for`.
- `ɵɵprojection(index)` — create a content-projection slot (`<ng-content>`).
- `ɵɵnamespaceSVG()` / `ɵɵnamespaceMathML()` — switch the DOM namespace for subsequent element creation.

Binding / update (update mode, mostly)

- `ɵɵproperty(name, value)` — set a DOM property (`el.value = x`), only if changed.
- `ɵɵattribute(name, value)` — set an HTML attribute via `setAttribute`.
- `ɵɵstyleProp` / `ɵɵclassProp` / `ɵɵstyleMap` / `ɵɵclassMap` — style/class bindings, resolved through the styling data structure (a separate "styling context" merges static, map, and individual bindings).
- `ɵɵtextInterpolate` / `ɵɵtextInterpolate1..8` / `ɵɵtextInterpolateV` — set a `Text` node's data from 1..8 (or N) interpolated expressions, using specialized fast paths per arg count to avoid array allocation.

- `ɵɵadvance(n)` — move the internal node cursor forward by `n` slots. Because bindings are emitted per-node in document order, the compiler doesn't need to re-specify "which element" for every binding — it just advances past however many nodes were skipped since the last binding.

Event handling (create mode: registers the listener once)

- `ɵɵlistener(eventName, handlerFn, useCapture?, eventTargetResolver?)` — attaches a native event listener wired to invoke the handler inside `ctx`, wrapped so it also marks the view (and ancestors, for the zone-less/OnPush "dirty" bit) so a subsequent CD pass will pick up any state the handler changed.

Container/view management

- `ɵɵtemplate` (as above) creates the container; the actual view instances inside it are created/destroyed imperatively by directive code (`NgIf`, `NgForOf`, the `@if`/`@for` block runtime) calling `ViewContainerRef` methods, **not** by more compiled instructions — those directives are themselves just consumers of the public/internal `ViewContainerRef` API.

Directive/pipe instantiation

- `ɵɵdirectiveInherit`, `ɵɵProvidersFeature`, `ɵɵpipe(index, pipeName)` — wire up directive matching and pipe instances into the same slot-indexed `LView`.

2.4 `TNode`, `TView`, `LView` — the backing data structures

Angular's rendering data is deliberately split into a "**static, shared per template**" part and a "**per-instance**" part, because most templates are instantiated many times (every row of an `*ngFor`, every embedded view of an `*ngIf`):

- `TView` ("template view") — created **once per template function**, shared by all instances. Holds the compiled shape: the array of `TNode`s (static node metadata: tag name, attrs, static styling, parent/child/next pointers, directive indices, injector index), the list of pipe/directive defs, `firstCreatePass` bookkeeping, and the queues of hook functions (`preOrderHooks`, `contentHooks`, `viewHooks`, etc.) precomputed on the very first creation pass so later instantiations skip that discovery work entirely.
- `TNode` — one static descriptor per element/text/container/i18n slot in the template. Contains the shape of the node (its position in the tree, its injector chain position) but **not** the actual DOM node or actual bound values — those live in the instance-level array.
- `LView` ("logical/live view") — created **once per instantiation** of a `TView`. It's a plain JS array (for perf: monomorphic array access, no property lookups) indexed in parallel with `TNode`s, holding: actual DOM node references, the component instance, directive instances, binding memoization slots, the `TView` back-reference, the parent `LView`, and flags like `Dirty`, `Attached`, `CheckAlways` (for `OnPush` tracking).
- `LContainer` — the runtime counterpart of a `ɵɵtemplate` slot: an array holding the anchor comment `RNode`, the parent `LView`, and a list of currently-inserted `LView`s (the embedded views for that `*ngIf`/`*ngFor` position).

The `TView`/`LView` split is why Angular's per-instance instantiation cost is low relative to the initial compile: structural work (matching directives, computing hook queues, building the `TNode` tree) happens once on `firstCreatePass`; every subsequent instantiation of that template just allocates a new `LView` array and runs the create-mode instructions to populate it, without re-discovering structure.

2.5 Embedded views, `ViewContainerRef`, and comment anchors

`*ngIf`, `*ngFor`, `@if`, `@for`, and `<ng-template>` all compile to a `ɵɵtemplate` instruction that:

1. Creates a **comment DOM node** (visible in DevTools as `<!--ng-container-->` or similar) as a stable, always-present anchor in the parent's DOM position.
2. Registers an `LContainer` at that slot, holding a reference to a nested `TView` (compiled from the structural directive's template content) generated as its own `X_Template` function with its own `decls`/`vars`.
3. Does **not** create any of the "inner" DOM by itself — that's deferred to whatever directive owns the container (`NgIf`, `NgForOf`, or the `@if`/`@for` runtime), which injects `ViewContainerRef` and imperatively calls:
    - `viewContainerRef.createEmbeddedView(templateRef, context, index?)` — instantiate the nested `TView` into a new `LView`, run its create-mode instructions (building real DOM nodes), and **insert those DOM nodes as siblings immediately after the anchor comment**.
    - `viewContainerRef.remove(index)` / `.clear()` — detach the `LView`, run `ngOnDestroy` on its directives/pipes, and remove its DOM nodes from the document.
    - `viewContainerRef.move(view, newIndex)` — reorder without destroy/recreate (this is what `NgForOf`'s `trackBy` unlocks: matching items by track key lets Angular **move** existing views/DOM instead of destroying and rebuilding them).

Because the container's position is a **comment node**, not an element, `*ngIf`/`*ngFor` never wrap your content in an extra `<div>` — the comment is invisible and purely a bookkeeping marker for "insert future sibling views right here."

`@if`/`@for` (the modern control-flow block syntax, Angular 17+) compile to the same underlying `ɵɵtemplate`/`LContainer`/embedded-view machinery, but via built-in instructions (`ɵɵconditionalCreate`/`ɵɵconditional`, `ɵɵrepeaterCreate`/`ɵɵrepeater`) rather than the `NgIf`/`NgForOf` directives, and `@for` mandates a `track` expression (no implicit identity fallback), plus it maintains an internal keyed diffing algorithm optimized for LCS-style move detection — generally more efficient than `NgForOf`'s default `IterableDiffer`.

2.6 Node injectors vs module injectors during rendering

Dependency injection during template rendering walks **two separate, connected injector hierarchies**, and the rendering engine is precisely what builds and threads through the first one:

- **Element/Node Injector hierarchy** — a tree that mirrors the DOM tree, built as `ɵɵelementStart`/`ɵɵtemplate` instructions execute. Every element (and every embedded view's host) that has at least one directive with providers gets an injector "slot" in `TNode`/`LView` — this is stored compactly as flat parallel arrays (not one `Injector` object per node) for memory efficiency, walked via `TNode.injectorIndex`/`parent` pointers. Providers registered via a directive/component's `providers`/`viewProviders` array live **here**. Resolution for a token requested inside a directive/component constructor walks: this node's providers → `viewProviders` of the hosting component (if resolving from inside its own template) → ancestor element injectors up to the root of the current view → **then** escalates to the module injector.
- **Module Injector hierarchy** — the traditional `Injector` tree built from `NgModule`/`providedIn:`

`'root'` /environment injectors, entirely separate from DOM structure. This is what backs services registered app-wide.

The **rendering engine's job** is to make the node-injector walk possible at all: as `ɵɵelementStart` builds each `TNode`, the compiler-emitted `ɵɵdirectiveInherit` /provider instructions record which providers live at that node, and `ɵɵtemplate`'s embedded views get their **own** injector chain rooted at the embedding component's node injector (not the *lexical* place in the source file) — meaning a directive's `ElementRef` /injected service resolution depends on where in the **rendered tree** it sits, not merely where its selector textually appears. This is why `providers` on a component with an `*ngIf` sibling directive can differ per structural-directive instantiation, and why `@Host()` / `@SkipSelf()` / `@Optional()` decorators are meaningful modifiers on this specific walk.

2.7 Rendering engine ↔ Change detection: they are the same walk

This is the single most commonly misunderstood point in interviews: **change detection does not "trigger" the rendering engine as a separate step — change detection *is* invoking the compiled template function in `Update` mode.**

Concretely:

1. Zone.js (or manual `ApplicationRef.tick()` in zoneless apps, or signals-based scheduling) decides *when* to run a CD pass.
2. `ApplicationRef.tick()` walks the tree of root `LView`s and for each, calls `refreshView()`.
3. `refreshView()` calls the component's `TView.template` function with `RenderFlags.Update` — literally re-executing `ɵɵproperty`, `ɵɵtextInterpolate`, `ɵɵadvance`, etc. — which is the **exact same instruction stream** that ran in `Create` mode, just with the create-only branch (`rf & Create`) skipped.
4. Each `ɵɵproperty` / `ɵɵtextInterpolate*` call does its own micro dirty-check (`bindingUpdated(lView, bindingIndex, value)`) comparing against the memoized slot from last time, and only touches the real DOM if the value changed — this is Angular's actual "diffing," done per-binding at the instruction level, not via a virtual DOM tree diff.
5. `refreshView()` also walks into every `LContainer` at this view's slots and recursively refreshes each embedded `LView` inside it (each `*ngFor` row, the current `*ngIf` / `@if` branch's view) — the CD tree walk **is** the LView tree walk, which **is** the same tree the rendering engine built.
6. `OnPush` components short-circuit this walk: `refreshView` checks `LView[FLAGS] & LViewFlags.CheckAlways` / the component's dirty flag before descending, so its subtree's `Update`-mode instructions are skipped entirely unless an `@Input` reference changed, an event originated inside it, `markForCheck()` was called, or (with signals) a read signal changed.

So "change detection" is best understood as: *the rendering engine's update-mode instruction execution, applied recursively down the `LView` tree that the rendering engine itself constructed during create mode.* There's one tree, one walk, two modes.

3. Code Examples

A sketch of what the Ivy compiler emits for a component like:

```
<h1>Hello, {{ name }}</h1>
@if (loggedIn) {
  <p>Welcome back, {{ name }}!</p>
} @else {
  <p>Please log in.</p>
}
```

```
// --- Main component template function ---
function MyComponent_Template(rf: RenderFlags, ctx: MyComponent) {
  if (rf & RenderFlags.Create) {
    ɵɵelementStart(0, "h1");
    ɵɵtext(1);                        // text node for interpolation, slot 1
    ɵɵelementEnd();
    ɵɵconditionalCreate(2, MyComponent_Conditional_2_Template, 1, 1)  // @if/@else
container, slot 2
      (MyComponent_Conditional_2_else_Template, 1, 0);
  }
  if (rf & RenderFlags.Update) {
    ɵɵadvance(1);                                    // move cursor to slot 1 (the text node)
    ɵɵtextInterpolate1("Hello, ", ctx.name, "");     // 1-arg fast-path interpolation
    ɵɵadvance(1);                                    // move cursor to slot 2 (the container)
    ɵɵconditional(ctx.loggedIn ? 0 : 1);             // pick branch 0 (@if) or 1 (@else), or
-1 for none
  }
}

// --- Embedded view template for the @if branch ---
function MyComponent_Conditional_2_Template(rf: RenderFlags, ctx: MyComponent) {
  if (rf & RenderFlags.Create) {
    ɵɵelementStart(0, "p");
    ɵɵtext(1);
    ɵɵelementEnd();
  }
  if (rf & RenderFlags.Update) {
    // ctx here is the *host* component's context, captured via closure binding,
    // NOT a separate scope object — @if/@for embedded views share the parent's ctx.
    ɵɵadvance(1);
    ɵɵtextInterpolate2("Welcome back, ", ctx.name, "!", "");
  }
}

// --- Embedded view template for the @else branch ---
function MyComponent_Conditional_2_else_Template(rf: RenderFlags, ctx: MyComponent) {
  if (rf & RenderFlags.Create) {
    ɵɵelementStart(0, "p");
    ɵɵtext(1, "Please log in.");
    ɵɵelementEnd();
  }
  // no Update block needed: nothing dynamic in this branch
}

// --- ComponentDef wiring ---
```

```
MyComponent.ɵcmp = ɵɵdefineComponent({
  type: MyComponent,
  selectors: [["app-my-component"]],
  decls: 3,   // h1(0), text(1), conditional-container(2)
  vars: 2,    // interpolation binding + conditional branch-index binding
  template: MyComponent_Template,
});
```

Notes embedded in the sketch:

- `ɵɵconditionalCreate` / `ɵɵconditional` are the actual Angular 17+ built-in-control-flow instructions; under the hood they manage an `LContainer` exactly like `ɵɵtemplate` did for `*ngIf`, but the branch index (not a boolean) drives which nested `TView` is instantiated, so `@if/@else if/@else` compiles to **one container with N candidate templates**, switching by index instead of one `NgIf` container per branch.
- The `@else` branch has no `Update` block at all — the compiler statically knows nothing in it is dynamic, so no `vars` are spent and no update-mode work happens for that branch when active.
- `ɵɵadvance` distances are relative to the previous cursor position, which is why reordering/adding a static node without a binding doesn't need every subsequent `ɵɵadvance` call to change value-by-value by hand — the compiler computes the deltas.

4. Internal Working — Step by Step

Step 1: First creation pass builds `TView` (once).
When `MyComponent` is instantiated for the very first time anywhere in the app, Angular calls `MyComponent_Template(Create | Update, ctx)` but with `tView.firstCreatePass = true`. During this pass, in addition to running instructions, Angular also populates `TNode` metadata (tag names, static attrs, directive matches, injector slot indices) and precomputes hook queues. This `TView` object is cached on `ComponentDef` and reused by every future instance — it is never rebuilt again for this component class.

Step 2: `ɵɵelementStart` / `ɵɵelementEnd` build the real DOM tree.
`ɵɵelementStart(0, "h1")`:

- Looks up (first pass) or reads (later passes) the `TNode` for slot 0.
- Calls the renderer abstraction (`Renderer2` /native renderer) to `document.createElement('h1')`.
- Stores the real `HTMLElement` in `lView[HEADER_OFFSET + 0]`.
- Appends it into its parent's DOM node (tracked via the "current parent" pointer maintained by the instruction runtime — this is why `elementStart` / `elementEnd` must be balanced, mirroring how the template nests).
- Pushes this `TNode` as the "current" node so any `ɵɵproperty` / `ɵɵlistener` calls before the matching `elementEnd` apply to it.

`ɵɵelementEnd()` pops the "current node" stack back to the parent, so subsequent sibling instructions attach correctly.

`ɵɵtext(1)` similarly creates a `Text` DOM node and appends it, storing the reference in `lView[HEADER_OFFSET + 1]`.

By the time create mode finishes, the **entire static DOM shape** for this view exists as real nodes, parented correctly, sitting in `lView`'s indexed slots — but bound values (interpolations, dynamic properties) are still whatever the initial synchronous update-mode pass (also run this first time) fills in.

Step 3: Embedded views (`*ngIf` / `*ngFor` / `@if` / `@for`) are inserted via comment-anchored containers.

- `ɵɵtemplate` (or `ɵɵconditionalCreate` / `ɵɵrepeaterCreate`) creates a `Comment` DOM node at that template position and an `LContainer` array structure referencing it, storing both in the parent `LView`'s slot.
- The nested template (`MyComponent_Conditional_2_Template`) is compiled as its **own mini `TView`**, lazily created on first use of that branch (cached thereafter, same as any `TView`).
- When update-mode logic decides a view should exist (`ɵɵconditional(0)`, or `NgForOf`'s diffing determining a row needs creating), Angular calls `createEmbeddedView`-equivalent logic: allocate a new `LView` for that nested `TView`, run its `Create` mode instructions (building its own DOM subtree), then **insert those new DOM nodes into the document immediately following the container's comment anchor** (`renderer.insertBefore` against the comment's `nextSibling`).
- The new `LView` is pushed into the `LContainer`'s views array, and critically, the `LContainer`'s slot in the *parent* `LView` now roots a subtree that the change-detection walk will descend into.
- Removing a view (`*ngIf` flips false, an `*ngFor` row is deleted, `@if` switches branch) does the reverse: run `ngOnDestroy` on that `LView`'s directive instances, remove its DOM nodes from the document, and splice it out of the `LContainer`'s views array — the comment anchor itself is never removed, since the container position must remain addressable for future insertions.

Step 4: The `LView` tree *is* the change-detection tree.
Every `LView` (component views and embedded views alike) has a pointer to its parent `LView` and, in the case of component `LView`s, is discoverable from `ApplicationRef.components` /the root view down through every `LContainer` it owns. `ApplicationRef.tick()`:

1. For each attached root component `LView`, calls `refreshView(lView, tView, context)`.
2. `refreshView` executes `tView.template(RenderFlags.Update, context)` — running exactly the update-mode instructions shown in Section 3.
3. It also iterates that view's registered containers (recorded during create mode) and recursively calls `refreshView` on each currently-inserted embedded `LView` — meaning a `*ngFor` with 500 rows means 500 recursive `refreshView` calls, each running that row template's own `Update` block.
4. For a **child component** encountered as a node in the current view (not an embedded view, an actual nested `<app-child>`), `refreshView` similarly finds the child's own `LView` (stored at that node's slot) and recurses into it, UNLESS that child is `OnPush` and not marked dirty, in which case the recursion into that subtree is skipped outright — no instructions of any kind execute for it this pass.

Step 5: Why this design makes `OnPush`/signals cheap.
Because "run change detection" literally means "walk this specific `LView` tree and call `Update`-mode functions," skipping a subtree is just *not calling the function* for that branch — there's no separate "diff tree A vs tree B" cost to avoid, because there was never a second tree. Signals-based reactivity (Angular 16+) layers a finer-grained mechanism on top: signal reads inside a template are tracked so only the `LView`s whose bindings actually depend on a changed signal get marked for refresh, letting Angular (in zoneless mode) schedule `tick()` only when something signal-driven actually changed, rather than on every browser macrotask Zone.js would otherwise intercept.

5. Edge Cases & Gotchas

- `ɵɵadvance` **is stateful and order-dependent.** Generated code assumes a specific sequential execution order matching the template's node order. Hand-editing generated output (never do this, but it clarifies the model) or a compiler bug that miscounts `decls` causes `ɵɵadvance` to point at the wrong slot — the classic symptom is a binding updating the wrong DOM node or an `ExpressionChangedAfterItHasBeenCheckedError` from stale indices.
- **Comment anchors are permanent, not toggled.** `*ngIf` / `@if` / `<ng-template>` containers always leave a comment node in the DOM even when no view is currently inserted — inspecting production DOM and seeing stray `<!--...-->` nodes around conditionally-rendered content is expected, not a leak.
- **Embedded views share the *host* component's context, not a nested scope object.** A `*ngFor` / `@for` embedded view's `ctx` argument passed to its template function is a distinct micro-context (`{ $implicit, index, count, ... }`) merged conceptually with the enclosing component instance via closures at the compiler level — but there is no prototype-chain scope inheritance like Angular.js. This is why `let-item` template variables are only visible inside that specific embedded view's template function, not magically available to sibling code.
- `trackBy` **(NgForOf) /** `track` **(**`@for`**) change whether views are recreated or moved.** Without a stable track key, Angular's default identity diffing may destroy and recreate every downstream `LView` (running `ngOnDestroy` / `ngOnInit` again, losing component-local state like focus, form control state, animations-in-flight) even when the *item* conceptually persisted, just because the diffing couldn't prove object identity survived a reorder. `@for` **requires** an explicit `track` expression, precisely because this class of bug was common enough with `NgForOf`'s optional `trackBy` that the new syntax made it mandatory.
- **Node injector resolution depends on rendered position, not source position.** A directive applied to the same host element as a component (e.g., `<app-child myDirective>`) sees the component's `viewProviders` unavailable to it (viewProviders are only visible to the component's *own* template's descendants, not to sibling directives on the host element) — a frequent "why can't my directive inject this service that's clearly provided right here" interview gotcha.
- `OnPush` **blocks the *walk*, not the DOM mutation for already-scheduled work.** If a `OnPush` component's update-mode instructions were already mid-execution when a `markForCheck()` fires from deep within the same tick, it doesn't "abort" — Angular's dirty flags are checked at each subtree's entry point at the start of the pass, so a genuinely mid-flight execution completes normally; the effect of `markForCheck()` is scoped to *ensuring the next check considers this subtree*, not interrupting the current one.
- `ɵɵtextInterpolate1..8` **vs** `ɵɵtextInterpolateV`: the compiler picks the fixed-arity instruction for ≤8 interpolated expressions and falls back to the variadic (array-allocating) version beyond that — a template with 9+ interpolations in one text node is measurably (if marginally) slower per check than the same content split across nodes, purely due to the array allocation `ɵɵtextInterpolateV` needs internally.
- **Detached views still exist as** `LView`**s.** `ViewContainerRef.detach()` (as opposed to `.remove()`) pulls a view's DOM out and unhooks it from the CD walk, but keeps the `LView` alive in memory for potential re-`.insert()` later — a common source of "phantom" memory retention if code detaches views and never re-inserts or destroys them.

6. Interview Questions & Answers

Q1: What is the difference between create mode and update mode in a component's template function?
A: Every compiled template function receives a `RenderFlags` bitmask (`Create`, `Update`, or both) and an `if (rf & RenderFlags.X)` guard splits its body accordingly. Create mode (run once per view instantiation) executes node-creation instructions (`ɵɵelementStart`, `ɵɵtext`, `ɵɵtemplate`, `ɵɵlistener`) that build the actual DOM tree and register event handlers. Update mode (run on every change-detection pass) executes binding instructions (`ɵɵproperty`, `ɵɵtextInterpolate*`, `ɵɵattribute`) that push current values into already-existing DOM nodes. The first invocation typically runs both flags together (build then immediately populate); every later invocation runs only `Update`.

Q2: Why does Angular use `decls` and `vars` counts on `ComponentDef`, and what happens if they're wrong?
A: `decls` tells the runtime how many slots to pre-allocate in the `LView` array for nodes/containers/pipes in this template; `vars` tells it how many binding-memoization slots to reserve for dirty-checking dynamic expressions. Both are computed by the compiler, not the developer, so mismatches only occur from compiler bugs or (very rarely) hand-modified generated code — the symptom is instructions reading/writing the wrong array index, producing corrupted DOM updates or thrown "index out of bounds"-style errors deep in render3 internals.
Interviewer intent: checking whether the candidate understands that `LView` is a flat array indexed by compile-time-known offsets, not a dynamically-keyed structure — this is core to why Ivy is fast (monomorphic array access) versus a hypothetical dictionary-based view model.

Q3: How does `*ngIf` actually get its content in and out of the DOM?
A: The compiler emits a `ɵɵtemplate` instruction at that position, which creates a `Comment` node (an anchor) and an `LContainer` registered in the parent view. `NgIf` itself is a directive that injects `ViewContainerRef` and `TemplateRef`; in its own `ngOnChanges`, based on the boolean input, it calls `viewContainerRef.createEmbeddedView(templateRef)` (inserting a newly created view's DOM immediately after the anchor comment) or `viewContainerRef.clear()` (destroying the view and removing its DOM). The comment node itself is never removed — it persists as the stable insertion point.

Q4: What's the difference between `TView`/`TNode` and `LView`?
A: `TView` and `TNode` are the **static, compile-time-shape** data structures, created once per template function on the very first instantiation and shared/reused by every subsequent instance of that same template (a component class, or a given `*ngFor` row template). `LView` is the **per-instance, live** array holding actual DOM references, directive instances, and current binding values for one particular instantiation. This split lets Angular skip repeating the expensive "figure out the shape and directive matches" work (`firstCreatePass`) for every new row of an `*ngFor` — only a lightweight `LView` array allocation and instruction execution happens per instance thereafter.

Q5: Explain what `ɵɵadvance(n)` does and why it exists.
A: It moves an internal "current binding index" cursor forward by `n` slots within the current view before the next binding instruction executes. Because update-mode instructions are emitted in document order and each targets "whatever node is currently selected," the compiler doesn't need to pass an explicit index to every `ɵɵproperty`/`ɵɵtextInterpolate` call — it just emits the delta since the last binding. This keeps generated code compact and avoids repeatedly re-resolving "which node am I updating" from scratch.

Q6: If change detection isn't a virtual-DOM diff, what is Angular actually comparing?
A: Each binding instruction (`ɵɵproperty`, `ɵɵtextInterpolate*`, `ɵɵattribute`, etc.) does its own micro-comparison via `bindingUpdated(lView, bindingIndex, newValue)`: it reads the previously memoized value out of that binding's reserved `vars` slot, compares by reference/`===`, and only if different does it touch the real DOM and overwrite the memoized slot. There's no tree-diff step and no synthetic DOM representation being compared — the "diffing" is a flat, per-expression memoization check that happens to be re-run on every pass because the whole template function re-executes.

Q7: Why can a directive on `<app-child someDirective>` fail to inject a service that `AppChildComponent` provides via `providers`?
A: `providers` (and `viewProviders`) registered in `@Component({providers: [...]})` are visible to the **node injector chain rooted at that component's own template's descendants** and to the component's own constructor, but a sibling directive placed on the *host* element sits at the same `TNode` — for `viewProviders` specifically, visibility is scoped to the component's view, not siblings on its host node; the directive would need the component to expose it via plain `providers` (element-injector-visible to same-node consumers) rather than `viewProviders`, or the directive needs to be a descendant inside the component's template, not a co-occupant of the host tag.
Interviewer intent: distinguishes candidates who've actually hit `NullInjectorError` in this exact configuration from those who only know the `providers` vs `viewProviders` definitions abstractly.

Q8: How does `@for`'s `track` expression change what the rendering engine does compared to no tracking at all?
A: Without a stable identity key, reordering/filtering a list forces Angular's diffing to treat items it can't positively match as removed-then-added, which means destroying the corresponding embedded `LView`s (running `ngOnDestroy`, discarding directive/component state, losing DOM-level state like input focus or CSS transition state) and creating fresh ones — full create-mode instruction execution per affected row. With `track` (mandatory in `@for`), Angular's keyed diffing algorithm can recognize "this item just moved" and call the equivalent of `ViewContainerRef.move()` — reordering the existing `LView`/DOM nodes in the document without destroying/recreating them, preserving component state and DOM identity, and skipping create-mode work entirely for unmoved/moved rows.

Q9: What actually happens, step by step, when `ApplicationRef.tick()` runs?
A: It iterates the registered root components' `LView`s and calls `refreshView` on each. `refreshView` invokes that view's `TView.template` function with `RenderFlags.Update`, executing every binding instruction in document order (using `ɵɵadvance` to navigate) and updating any DOM whose memoized binding value changed. It then descends into every `LContainer` registered in that view (from `*ngIf`/`*ngFor`/`@if`/`@for`/`<ng-template>` usages) and recursively `refreshView`s each currently-inserted embedded view, and separately descends into any child component `LView`s found at their node slots — skipping that recursive descent entirely for `OnPush` children whose dirty flag isn't set. Lifecycle hooks queued in the `TView` (e.g. `ngAfterViewChecked`) fire at the appropriate points in this same walk.

Q10: Why is it accurate to say "the rendering engine and change detection are the same mechanism"?
A: Because there's no separate "CD engine" invoking or diffing against the "rendering engine" as a distinct subsystem — change detection *is* the act of calling the exact same compiled template function that built the DOM, just with the `Update` half of its `if (rf & flag)` branches executing instead of the `Create` half, walked recursively across the same `LView` tree that create mode itself constructed and linked together via `LContainer`s and child-component slots. There's one tree and one instruction-execution model; "running change detection" and "re-invoking update-mode rendering instructions across the LView tree" are the same sentence said two ways.

Interviewer intent: this question filters for genuine internals understanding versus memorized terminology — many candidates can define "zone.js triggers change detection" without ever having connected that to what actually executes underneath.

Q11: What's the practical difference between `ViewContainerRef.detach()` and `.remove()` / `.clear()`?
A: `remove()` / `clear()` destroy the targeted `LView`(s) outright: run `ngOnDestroy` on their directive/pipe instances, remove their DOM nodes from the document, and drop them from the `LContainer`'s tracked views — the memory is eligible for GC. `detach()` only unhooks a view from DOM presence and the change-detection walk (pulling its root DOM nodes out and excluding it from `refreshView` recursion) while keeping the `LView` object alive and reusable, so it can later be re-`insert()`ed (e.g., cheaply reinserted into a different container, common in virtual-scroll / view-recycling implementations like CDK's `VirtualScrollViewport`) without re-running create-mode instructions.

Q12: Why does an `@if` / `@else if` / `@else` chain compile to one container instead of one per branch, unlike historical `*ngIf; *ngIf: else`?
A: The built-in control-flow syntax compiles to `ɵɵconditionalCreate` (registering all candidate branch templates against a single `LContainer`/anchor) and `ɵɵconditional(branchIndex)` in update mode, which simply swaps which single nested `TView` is instantiated at that one container based on an integer index (or removes the view entirely for index `-1`). This avoids each branch needing its own separate comment anchor/`LContainer` and its own directive instance coordinating an if/else pair via `ElseRef`-style template references, reducing per-branch bookkeeping overhead and simplifying the generated code to a single switch-like update instruction.

Q13: Can two different structural directives on the same element (e.g., stacking `*ngIf` and `*ngFor` without `<ng-container>` / `<ng-template>`) both compile correctly?
A: No — each structural directive shorthand desugars to its own nested `<ng-template>` wrapping the element, so a single host element can only carry one structural directive shorthand directly; stacking two on the same tag is a compile error. To combine them you nest explicit `<ng-template>`s (or use `<ng-container *ngIf="...">` wrapping `<ng-container *ngFor="...">`), producing two separate `ɵɵtemplate` containers, one nested inside the other, each with its own `LContainer`/anchor comment.

Q14: How does the rendering engine handle content projection (`<ng-content>`), and why can't projected content be change-detected independently of its original context?
A: `ɵɵprojection(index)` marks a slot where nodes captured from the host component's usage site should be moved into. Critically, projected DOM nodes are **not** re-parented into a new `LView` — their bindings still belong to, and are refreshed as part of, the `LView` of the component that originally declared them (the parent/consumer), not the component doing the projecting. This is why a projected `{{ expression }}` still resolves against the consumer's context and why `OnPush` on the *projecting* component doesn't gate re-evaluation of bindings that live in the *projected* content's own originating view.

7. Quick Revision Cheat Sheet

- **Template function** = `(rf: RenderFlags, ctx) => { if (rf & Create) {...}; if (rf & Update) {...} }` — one function, two modes, driven by a bitmask.
- `decls` / `vars` — compile-time counts sizing the `LView` array's node slots and binding-memoization slots respectively.
- **Create-mode instructions**: `ɵɵelementStart/End`, `ɵɵelement`, `ɵɵtext`, `ɵɵtemplate`, `ɵɵlistener`, `ɵɵprojection` — build DOM + wire events, run once.

- **Update-mode instructions**: `ɵɵproperty`, `ɵɵattribute`, `ɵɵtextInterpolate*`, `ɵɵstyleProp`/`ɵɵclassProp` — push bound values into existing DOM, run every CD pass, each self-memoizing via `bindingUpdated`.
- `ɵɵadvance(n)` — moves the binding cursor forward `n` slots; document-order dependent.
- `TView`/`TNode` = static, compiled-once, shared shape. `LView`/`LContainer` = per-instance, live data (real DOM refs, directive instances, current values).
- **Embedded views** (`*ngIf`, `*ngFor`, `@if`, `@for`, `<ng-template>`) = a `ɵɵtemplate`-created `Comment` anchor + `LContainer`; actual insertion/removal is imperative, via `ViewContainerRef.createEmbeddedView/remove/clear/move`, not more compiled instructions.
- `track`/`trackBy` — enables `.move()` (preserve `LView`/DOM/state) instead of destroy+recreate on reorder.
- **Node injector tree** — mirrors rendered DOM/view structure, built as `ɵɵelementStart`/`ɵɵtemplate` execute; resolves `providers`/`viewProviders` by rendered position, escalates to the **module injector tree** only after exhausting node injectors.
- **CD = rendering engine, Update mode, recursive `LView` walk.** `ApplicationRef.tick()` → `refreshView()` per root `LView` → re-run `Update`-mode instructions → recurse into `LContainer`s and child-component `LView`s → skip subtree entirely if `OnPush` and not dirty.
- **No virtual DOM diff** — "diffing" is per-binding memoized-value comparison (`bindingUpdated`) inlined into each update instruction call.
- Comment anchors from structural directives are **permanent** in the DOM; only the views inside them come and go.

Created By - Durgesh Singh